MQTT, CoAP and HTTP Under Constrained Networks: A Reproducible Benchmark with JSON and CBOR Encodings

Divij Karanam
Independent Researcher

***Abstract*—IoT devices often communicate over slow, lossy links, yet protocol comparisons are frequently run on clean networks. We present a reproducible benchmark of MQTT, CoAP and HTTP with JSON and CBOR payloads under five emulated network profiles (baseline, 4G-like, 3G-like, 2G-like and 10% loss). Across 18,000 measured messages (three repetitions per configuration, plus 3,600 calibration messages), all three protocols delivered every message, but their costs differed sharply. CoAP needed 185 bytes on the wire per message versus 630 for HTTP (3.4× less) and tracked the link's one-way delay most closely. With 10% packet loss, however, CoAP's fixed 2–3 s retransmission timer produced a 95th-percentile latency of 2,670 ms, compared with 78 ms for HTTP and 382 ms for MQTT, and 7 of 1,200 CoAP messages were delivered but never acknowledged within 15 s. CBOR reduced payload size by 18% but total traffic by only 3–11%. We also document configuration pitfalls, including missing keep-alive, split writes interacting with Nagle's algorithm, and host-side emulation artifacts, which initially made HTTP appear about four times slower than it is. Code and data are released.**

***Index Terms*—Internet of Things, MQTT, CoAP, HTTP, CBOR, network emulation, benchmarking**

I. Introduction

Many IoT deployments, such as agricultural sensors, utility meters and environmental monitors, rely on cellular links with high latency, limited bandwidth and packet loss. Choosing an application protocol for such devices is a practical decision with consequences for latency, data cost and energy. The three most common choices are MQTT [1], a publish/subscribe protocol over TCP with a broker; CoAP [2], a REST-style protocol over UDP with optional confirmable messages; and HTTP/1.1 [3].

Earlier comparisons [4]–[6] established that the protocols trade reliability against overhead differently. We contribute: (i) a fully scripted benchmark that runs on a single Linux machine and emulates network impairments at the packet level; (ii) measurements of latency, tail latency, delivery, bytes on the wire and receiver CPU for the three protocols with JSON [7] and CBOR [8] payloads under five network profiles; and (iii) a documented set of configuration and measurement pitfalls that can distort such comparisons by large factors.

II. Related Work

Thangavel et al. [4] compared MQTT and CoAP through a common middleware and found that MQTT had lower delay at low loss rates but higher delay than CoAP at high loss rates, while CoAP generated less extra traffic for small messages. Naik [5] compared MQTT, CoAP, AMQP and HTTP qualitatively across criteria such as message size, overhead and quality of service. Gündoğan et al. [6] measured NDN, CoAP and MQTT on constrained IoT hardware. Our work complements these studies with an emulated-profile methodology, payload-encoding comparisons, tail-latency analysis and an explicit account of measurement pitfalls.

III. Testbed and Method

A. Protocol configurations

Each protocol has a simulated sensor node that sends 200 readings (temperature, humidity, PM2.5, battery voltage; about 107 bytes in JSON) sequentially, with 50 ms between messages. **HTTP**: a Flask application served by waitress [9], one persistent keep-alive connection, `TCP_NODELAY` on both ends, and each request sent in a single write. **CoAP**: aiocoap [10] with confirmable POST requests and default RFC 7252 transmission parameters. **MQTT**: a Mosquitto 2.0 broker [11] and paho-mqtt clients [12] at QoS 1, with `TCP_NODELAY` on clients and broker. The receiver subscribes before measurement starts and signals readiness only after the broker confirms the subscription.

B. Network emulation

Impairments are applied to the benchmark ports on the loopback interface by a userspace emulator built on Linux NFQUEUE, used because the test kernel lacks `netem` [13]. It applies normally distributed delay jitter, random loss and a rate limit on a single FIFO queue that never reorders packets. Like `netem` on loopback, it impairs both directions, so the round-trip delay is about twice the configured delay. Table I lists the profiles. They are illustrative approximations of typical mobile conditions, not measurements of a specific operator. A calibration profile with the emulator active and zero delay added only 0.18–0.23 ms to median latency.

C. Metrics

One-way latency: the sender embeds a timestamp, and the receiver subtracts it on arrival (sender and receiver share a clock). For MQTT this spans sensor → broker → subscriber. *RTT*: from send to the protocol-level acknowledgment (HTTP response, CoAP ACK, MQTT PUBACK from the broker). *Delivery*: unique message IDs received divided by those

TABLE I
EMULATED NETWORK PROFILES (APPLIED IN EACH DIRECTION).

Profile	Delay (jitter)	Loss	Rate
Baseline	–	–	–
4G-like	25 ms (±5)	0.5%	–
3G-like	100 ms (±20)	1.5%	1 Mbit/s
2G-like	250 ms (±50)	3%	100 kbit/s
Lossy	10 ms	10%	–

sent. *Bytes on the wire*: change in the loopback interface's transmitted-byte counter during a run divided by the number of messages, including headers, TCP acknowledgments and retransmissions. For MQTT this counts both hops. *Receiver cost*: CPU time and peak resident memory of the receiving processes (broker plus subscriber for MQTT). Each configuration (5 profiles × 3 protocols × 2 encodings) ran three times. The host had 4 vCPUs (Intel Xeon, 2.8 GHz) and ran Linux 6.18 and Python 3.13.

IV. MEASUREMENT PITFALLS

The first version of the benchmark reported a median HTTP latency of about 440 ms on the 3G-like profile, four times the link delay. Packet captures revealed three causes, none inherent to HTTP. First, Flask's development server closes the connection after every response, adding a TCP handshake to each message. Second, the `requests` library writes headers and body separately, and Nagle's algorithm [14] then delays the body by one round trip until the headers are acknowledged. Third, because the emulator delays packets on the sending host, Linux TCP autocorking held small writes while an earlier packet was still queued locally, which would not happen on a real network. We disable autocorking during emulated runs. After these fixes, HTTP's median on the same profile was 115 ms.

Two further issues affected individual runs. Downloads made through a localhost proxy during the benchmark inflated the loopback byte counter, so every run that overlapped such activity was discarded and repeated. A receiver that announced readiness before its MQTT subscription was confirmed lost the first message in one lossy run; the receiver was fixed and all lossy MQTT runs were repeated. Both sets of superseded data are released alongside the final results. We recommend that protocol benchmarks report connection reuse, `TCP_NODELAY` settings and the location of emulation.

V. RESULTS

A. Baseline and bytes on the wire

On the unimpaired baseline, median one-way latency was 0.9 ms (MQTT), 1.1 ms (HTTP) and 1.3 ms (CoAP), so the protocol barely matters on a good network. Overheads differ substantially (Fig. 1). CoAP required about 185 bytes per message (one request and one acknowledgment datagram), HTTP about 630 and MQTT about 701. MQTT's figure covers two TCP hops, so per hop it is a little over half of HTTP's.

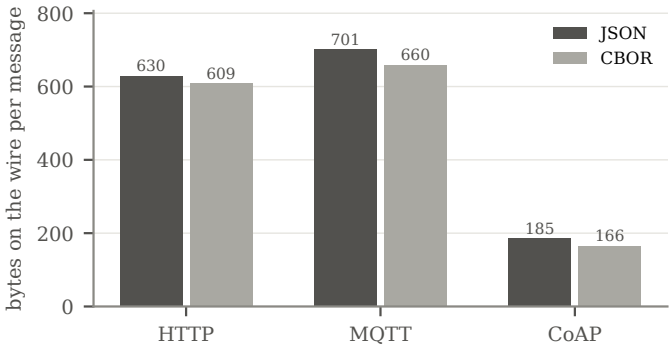


Fig. 1. Bytes on the wire per message on the baseline profile, including headers and acknowledgments (MQTT includes both broker hops).

TABLE II
RESULTS WITH JSON PAYLOADS (MEANS OVER THREE REPETITIONS).

Profile	Protocol	Median (ms)	p95 (ms)	Median RTT (ms)	Bytes/ msg
Baseline	CoAP	1.3	2.4	1.6	185
	HTTP	1.1	1.9	1.7	630
	MQTT	0.9	1.7	0.8	701
4G-like	CoAP	27.2	35.9	53.1	186
	HTTP	29.5	38.2	58.3	633
	MQTT	52.2	66.1	55.0	703
3G-like	CoAP	105	140	206	187
	HTTP	115	150	227	632
	MQTT	210	531	218	713
2G-like	CoAP	270	415	523	191
	HTTP	307	809	606	640
	MQTT	562	1,485	579	721
Lossy	CoAP	11.8	2,670	22.6	200
	HTTP	11.6	77.6	22.6	634
	MQTT	21.9	382	22.0	723

B. Latency on constrained links

Fig. 2(a) and Table II show that CoAP and HTTP medians follow the link's one-way delay: on the 4G-, 3G- and 2G-like profiles CoAP took 27, 105 and 270 ms, and HTTP took 30, 115 and 307 ms. The HTTP–CoAP gap grows from about 2 ms to 37 ms as the link worsens, plausibly because TCP acknowledgments queue ahead of data on the jittered link. MQTT took roughly twice as long (52, 210 and 562 ms) because both hops crossed the impaired link in our topology. The sender's wait for the broker's acknowledgment was similar to the other protocols (median RTT 55 ms versus 53 ms for CoAP on 4G-like), so in deployments where the broker sits next to the consumers, the second hop is short.

C. Packet loss and retransmission

All protocols delivered 100% of messages on every profile, because each provides reliability (TCP retransmission, CoAP confirmable messages, MQTT QoS 1). Loss therefore appears as delay. With 10% loss, medians stayed near the link delay, but the 95th percentile was 78 ms for HTTP, 382 ms for MQTT and 2,670 ms for CoAP (Fig. 2(b)). Linux TCP adapts its retransmission timing to the measured round-trip time and

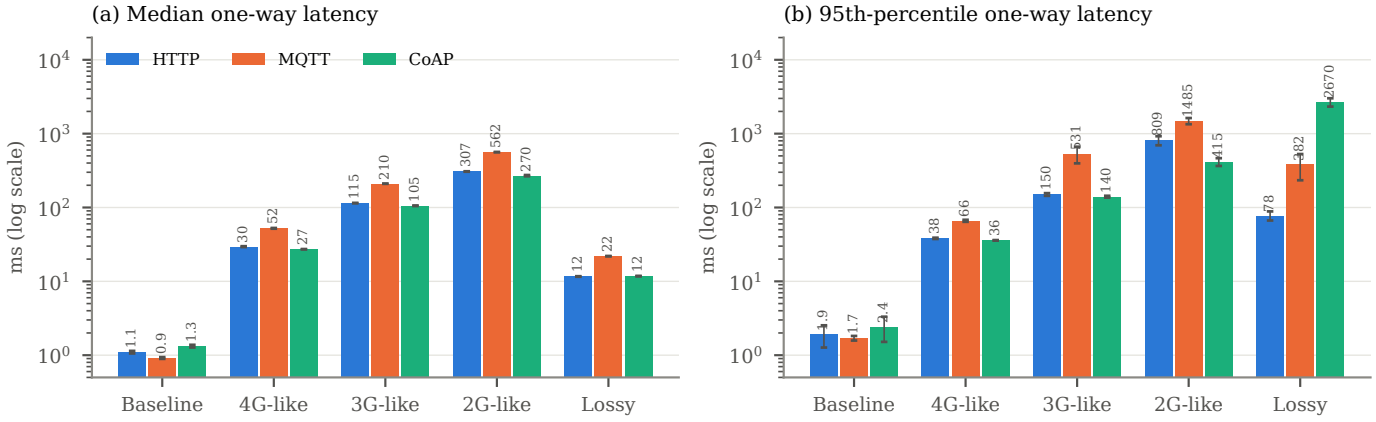


Fig. 2. One-way latency with JSON payloads (mean over three repetitions, error bars show standard deviation; log scale). MQTT latency includes the broker hop.

uses tail-loss probes [15], [16], so a lost segment on this 20 ms path is typically recovered within tens of milliseconds. CoAP's default initial timeout is 2–3 s regardless of the path [2]. In addition, 7 of the 1,200 CoAP messages on this profile reached the receiver but were never acknowledged to the sender within 15 s, so a real device would have treated them as failed. On the 2G-like profile (3% loss), in contrast, CoAP had the lowest 95th percentile (415 ms versus 809 ms for HTTP and 1,485 ms for MQTT), because fewer than 5% of messages needed a retransmission. On that profile, however, CoAP senders waited up to about 2.8 s (95th-percentile RTT) for acknowledgments that had been lost. Tail latency on the 2G-like profile also varied considerably between repetitions.

D. JSON versus CBOR

CBOR reduced the payload from about 108 to 89 bytes (−18%). Total bytes on the wire fell by 11% for CoAP, 6% for MQTT and 3% for HTTP, since headers and acknowledgments dominate small messages. CBOR had no consistent effect on latency.

E. Receiver cost

Receiver CPU per message was about 0.6 ms for MQTT (C broker plus Python subscriber), 0.9 ms for CoAP and 1.3 ms for HTTP, with peak memory of 30, 24 and 33 MB. These numbers reflect the specific implementations rather than the protocols themselves.

VI. Discussion and Limitations

For battery- or data-constrained devices on poor links, CoAP minimizes bytes and median latency, but its retransmission parameters should be tuned where loss is common. MQTT suits fan-out to many consumers, and broker placement determines its latency. HTTP is competitive once connections are reused and requests are sent in a single write. CBOR is most useful where protocol overhead is already small, as with CoAP.

All measurements used one machine, loopback networking and emulated impairments rather than radios or operator networks, and impairments were applied at the sending host. Messages were small and sent one at a time, so throughput effects of larger transfers were not studied. Implementations were Python-based (except Mosquitto) rather than embedded firmware.

VII. Conclusion

Under constrained networks, CoAP is the leanest protocol and the fastest in the median, HTTP is close behind when configured correctly, and MQTT's latency depends on broker placement. Under heavy loss, CoAP's default retransmission timer dominates its tail latency. Above all, configuration details can shift protocol comparisons by factors of four, so benchmarks should report them.

Data Availability

Code, raw data and the network emulator are available in the Zenodo record of this preprint.

Acknowledgment

The author used Claude (Anthropic), an AI assistant, to help write the benchmark code, run the experiments and draft this manuscript. The author is responsible for the content of the paper.

References

[1] A. Banks and R. Gupta, "MQTT version 3.1.1," OASIS Standard, 2014.
[2] Z. Shelby, K. Hartke, and C. Bormann, "The constrained application protocol (CoAP)," RFC 7252, 2014.
[3] R. T. Fielding, M. Nottingham, and J. Reschke, "HTTP/1.1," RFC 9112, 2022.
[4] D. Thangavel, X. Ma, A. Valera, H.-X. Tan, and C. K.-Y. Tan, "Performance evaluation of MQTT and CoAP via a common middleware," in *Proc. IEEE Int. Conf. Intelligent Sensors, Sensor Networks and Information Processing (ISSNIP)*, 2014.
[5] N. Naik, "Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP," in *Proc. IEEE Int. Systems Engineering Symposium (ISSE)*, 2017.
[6] C. Gündoğan, P. Kietzmann, M. Lenders, H. Petersen, T. C. Schmidt, and M. Wählisch, "NDN, CoAP, and MQTT: A comparative measurement study in the IoT," in *Proc. ACM Conf. Information-Centric Networking (ICN)*, 2018.

[7] T. Bray, “The JavaScript object notation (JSON) data interchange format,” RFC 8259, 2017.
[8] C. Bormann and P. Hoffman, “Concise binary object representation (CBOR),” RFC 8949, 2020.
[9] Pylons Project, “waitress: a production-quality pure-Python WSGI server,” https://github.com/Pylons/waitress.
[10] C. Amsüss, “aiocoap: The Python CoAP library,” https://github.com/chrysn/aiocoap.
[11] R. A. Light, “Mosquitto: server and client implementation of the MQTT protocol,” *Journal of Open Source Software*, vol. 2, no. 13, p. 265, 2017.
[12] Eclipse Foundation, “Eclipse Paho MQTT Python client,” https://github.com/eclipse-paho/paho.mqtt.python.
[13] S. Hemminger, “Network emulation with NetEm,” in *Proc. Linux Conference Australia (linux.conf.au)*, 2005.
[14] J. Nagle, “Congestion control in IP/TCP internetworks,” RFC 896, 1984.
[15] V. Paxson, M. Allman, J. Chu, and M. Sargent, “Computing TCP’s retransmission timer,” RFC 6298, 2011.
[16] Y. Cheng, N. Cardwell, N. Dukkipati, and P. Jha, “The RACK-TLP loss detection algorithm for TCP,” RFC 8985, 2021.